Automatic Program Generation for a Sheet-Metal Handling Robot in Robotic Press Brake Bending

Sooraj Sudhakaran

***Abstract*—Robotic press brake bending can reduce manual handling effort in sheet-metal production, but program creation remains a major barrier when workpieces, orientations, and bending sequences change. Conventional workflows often require manually defined waypoints, fixed handling sequences, and operator-dependent reprogramming for new part variants. This paper presents a simulation-based automatic robot program generation method for a sheet-metal handling robot in front of a press brake. The method integrates orientation selection, constraint-based suction grip planning, automatic regrip handling, opposite-face bending support, and trajectory generation in a modular planning pipeline. The working hypothesis is that a configuration-driven rule-based planner can reduce manual programming effort while maintaining feasible execution for representative bending scenarios. The system is evaluated in a virtual robotic cell using scenarios with varying sheet dimensions, arbitrary initial orientations, regripping requirements, and opposite-face bending operations. The results show that executable robot programs can be generated without manual waypoint definition. Compared with a manual drag-and-drop programming workflow, adaptation after geometry or pose changes is reduced from several minutes to a short automatic recalculation step. The findings indicate that rule-based program generation is a practical intermediate approach between manual teaching and full CAD-based offline programming for flexible robotic bending cells.**

***Index Terms*—Automatic robot programming, press brake bending, robotic sheet-metal handling, suction grip planning, regrip planning, industrial robotics.**

I. Introduction

Press brake bending is a core manufacturing process for sheet-metal components. In industrial practice, the automation of this process requires coordination between the bending machine, the robot, the gripper, and the workpiece. Robotic cells can reduce repetitive manual handling, but they also introduce a programming burden because the robot must approach, grip, move, reorient, and release the sheet under changing geometric and kinematic constraints. This burden is particularly relevant for low-volume and variant-rich production, where a change in sheet size or initial pose can require manual modification of several motion steps.

Manual and teach-based robot programming remain common in industrial environments, despite the increasing availability of offline programming and simulation tools. Recent reviews of industrial robot programming show that programming effort, usability, and adaptability still limit the deployment of flexible robotic systems in small-batch and variant-rich production [1]. In robotic press brake bending, the problem is more specific: the geometry and handling state of the sheet can change after each bend, and the selected grip may become infeasible for a later operation. Prior work has addressed sheet-metal bending process planning [2], robotic bending cells [3], and robot structures for sheet-metal bending [4]. However, a practical gap remains between high-level process planning and the automatic generation of executable handling programs for a specific robotic runtime.

The research problem addressed in this paper is the lack of a systematic method for generating robot programs for press brake sheet-metal handling when workpiece orientation, grip feasibility, regripping, and opposite-face bending must be considered together. The novelty lies in combining these planning functions into a modular, configuration-driven pipeline that generates executable robot motion logic without manually defining waypoints for every new workpiece condition.

The working hypothesis is that a deterministic rule-based program generator can reduce manual programming effort while maintaining feasible simulation execution for representative robotic press brake bending scenarios. The scope is limited to simulation-based validation, rectangular sheet geometries, vacuum-based gripping, and predefined cell configuration. Physical robot deployment, variable bend angles, full collision-free motion planning, and perception-based object detection are outside the present scope.

II. State of the Art / Related Work

Robot programming approaches can be broadly grouped into manual teaching, offline programming, skill-based programming, and automatic or semi-automatic task-level generation. Manual approaches provide direct control but require repeated waypoint definition and operator expertise. Offline programming can reduce downtime by using virtual models, but accurate cell modelling and post-processing remain necessary for industrial deployment [1].

Sheet-metal bending has been studied from the viewpoint of process planning and sequencing. Gupta et al. described automated process planning for sheet-metal bending operations, including the need to consider tooling, bend sequencing, and manufacturability constraints [2]. Aomura and Koguchi proposed optimized bending sequence planning for sheet-metal parts handled by a robot, including grasping positions and repositioning considerations [3]. More recent work by Xu et al. proposed and evaluated a six-axis Cartesian coordinate robot for sheet-metal bending, confirming continued research interest in specialized robotic architectures for this process [4].

Digital twin and offline programming methods are increasingly used to validate industrial systems before deployment.

The author is with Deggendorf Institute of Technology, Campus Cham.

Their value lies in early testing, visualization, and process verification, but the modelling effort can be high when the complete cell, tooling, and workpiece state must be represented accurately. Modern digital twin reviews describe this trade-off between virtual modelling capability and implementation complexity in industrial systems [5]. For the present work, a lighter rule-based generator is selected because the goal is not to reproduce a complete CAD-based planning environment, but to generate executable handling logic from constrained configuration data.

Modern robotic software research emphasizes modularity, middleware abstraction, and reproducible deployment. ROS 2, for example, has been developed to support more scalable and reliable robotic software architectures in real-world systems [6]. Although the present implementation is not a ROS 2 system, the same architectural principle is relevant: planning logic should remain separated from the execution interface so that the system can be maintained and extended.

Grip planning is another relevant research area. Vacuum-based end effectors are widely used for industrial sheet and panel handling because they can grip flat surfaces with relatively simple mechanics. Learning-based suction grasping methods such as Dex-Net 3.0 demonstrate how suction targets can be evaluated using geometric and physical criteria [7]. The present system does not implement a learning-based grasp planner; instead, it uses deterministic geometric and holding-force constraints suited to a controlled industrial sheet-metal scenario.

Trajectory generation for industrial robots must consider smoothness, feasibility, and compatibility with the robot controller. Recent work on industrial robot velocity profiles and interpolation methods illustrates the continuing importance of smooth path execution for industrial motion quality [8]. In the proposed pipeline, this requirement is addressed through structured waypoint generation and smooth motion profiles rather than global optimal path planning.

III. Methodology

The proposed method follows a modular planning pipeline. The input consists of workpiece geometry, sheet mass assumptions, bend-side requirements, workspace zones, suction cup parameters, and robot execution configuration. The output is an executable robot program sequence for the virtual press brake cell. The method is designed for deterministic behavior rather than statistical optimization, because repeatability and traceability are important for industrial commissioning.

The system architecture contains five functional layers. The input and configuration layer stores sheet dimensions, hole data, bend information, gripper parameters, workspace zones, and interface settings. The geometry and constraint evaluation layer checks the admissibility of sheet orientation, suction cup placement, edge clearance, hole avoidance, and holding force. The planning and decision layer determines the required process flow, including direct bending, regripping, or opposite-face handling. The trajectory layer converts selected process states into Cartesian waypoints and motion commands. The validation layer prevents execution when the generated state violates predefined reachability, pose, or process constraints.

Table I summarizes the module responsibilities. The table is included to make the transformation from input data to executable program logic explicit.

TABLE I
Planning Pipeline Modules

Module	Function
Configuration	Stores sheet, gripper, workspace, and execution parameters.
Orientation	Selects an admissible pickup and handling orientation.
Grip planning	Generates and validates suction cup candidates.
Regrip logic	Inserts reorientation steps when reachability changes.
Trajectory	Converts process states into executable waypoints.
Validation	Blocks unsafe or infeasible commands before execution.

Orientation selection is performed before gripping and bending. If the initial sheet pose is admissible, it is preserved. If the pose violates zone or orientation rules, corrective rotations and fallback configurations are evaluated. This procedure avoids unnecessary manual adjustment when the workpiece is placed in different initial orientations.

Grip planning is based on deterministic candidate generation. Candidate suction points are generated on feasible sheet regions and rejected when they violate edge clearance, spacing, hole exclusion, or simplified holding-force constraints. Multi-cup configurations are preferred when they improve stability. The selected grip is then passed to the handling and trajectory modules.

Regrip planning is triggered when a later bending pose cannot be reached or handled with the current grip state. The planner inserts an intermediate placement and reorientation sequence, after which a new valid grip is selected. Opposite-face bending is handled by applying a structured sheet-flip workflow before continuing with the same validation and trajectory generation logic.

The trajectory generator produces staged Cartesian motion segments, including approach, process, and retreat phases. The use of structured waypoints reduces dependence on manually taught positions. The system remains deterministic: identical configuration inputs produce identical program sequences, which supports repeatability and debugging.

The evaluation uses a virtual robotic press brake cell. The test scenarios include variations in sheet dimensions and mass assumptions, arbitrary initial orientations, different workspace regions, cases requiring regrip operations, and tasks requiring both primary-side and opposite-face bending. The main metrics are execution success in simulation, planning feasibility, regrip occurrence, adaptation time after input changes, and comparison with a manual programming workflow.

The evaluation protocol is restricted to functional validation. A scenario is considered successful when the generated program reaches the required process states in simulation, applies the selected grip and release sequence, and completes the planned bending-side workflow without manual waypoint correction. A scenario is considered rejected rather than failed

when a constraint violation is detected before execution and no feasible recovery is available under the defined assumptions.

IV. Results

The implemented system generated executable robot programs in the evaluated virtual scenarios without requiring manual waypoint definition. The orientation module produced valid pickup orientations for arbitrary initial sheet poses within the tested workspace assumptions. When an initial orientation already satisfied the admissibility rules, the pose was preserved. When it did not, the correction strategy produced an admissible alternative. This result confirms that input pose variation can be handled by the planning logic rather than by manually editing robot targets.

The grip planner generated suction-cup configurations that satisfied the implemented edge-clearance, hole-avoidance, spacing, and holding-force checks. Configurations violating these constraints were rejected before execution. In the tested multi-cup cases, the selected positions maintained the required geometric separation and remained outside internal hole regions. These results show only compliance with the implemented model; they do not prove real vacuum reliability under leakage, surface contamination, or dynamic loading.

The regrip module identified scenarios in which the current grip state was not suitable for later bending operations and inserted an intermediate reorientation sequence. The opposite-face workflow completed the sheet flip and continued the bending sequence using the same planning and validation modules. No separate manually programmed waypoint chain was required for these cases.

Table II summarizes the observed comparison between the conventional manual programming workflow and the automated program generation workflow used in the simulation environment.

The planning computation time remained lower than the simulated robot execution time in the evaluated cases. Error conditions such as infeasible grip configurations and unreachable poses were detected before unsafe execution commands were issued in the virtual environment.

From an engineering perspective, the most important result is not only the reduction in adaptation time, but the change in task type. In the manual workflow, the operator modifies a program by selecting or editing motion blocks and waypoints. In the automated workflow, the same change is represented as an input or configuration update, after which the program is regenerated. This reduces dependence on low-level robot programming knowledge.

V. Discussion

The results support the working hypothesis within the defined simulation scope. A configuration-driven rule-based planner can reduce manual programming effort for representative robotic press brake bending scenarios while maintaining feasible virtual execution. The strongest effect is observed after workpiece geometry or pose changes: instead of redefining robot waypoints, the operator changes input parameters and the planner recalculates the sequence.

The approach differs from full CAD-based offline programming. It does not require a complete CAD-integrated digital model of every cell element, but it also does not provide the same level of global collision checking and geometry generality. It is therefore best interpreted as an intermediate automation layer for structured bending cells with known constraints. This position is relevant for industrial environments where a full digital twin may be desirable but too expensive or time-consuming for an early-stage automation task.

The suction grip planner is suitable for the rectangular sheet and circular-hole assumptions used in the evaluation. Compared with learning-based suction grasping research [7], the proposed planner is simpler and more transparent, but less general. This trade-off is acceptable for the targeted controlled cell, where interpretability and deterministic behavior are valuable.

Several limitations remain. The evaluation is simulation-based and does not include calibration error, sensor noise, vacuum leakage, tool wear, or dynamic sheet deformation. The geometry model is limited to rectangular sheets, a fixed bend-angle assumption, and predefined workspace zones. Collision detection is not comprehensive, and the trajectory planner does not optimize globally for time, energy, or clearance. These limitations prevent direct claims about physical production readiness.

The results should therefore be interpreted as a proof of functional feasibility rather than a final validation of industrial robustness. A physical deployment would require calibration between the virtual and real cell, verification of the tool center point, validation of vacuum force margins, and safety review according to the target installation.

Future work should include physical robot validation, perception-based pose estimation, variable bend angles, generalized sheet geometry, force or vacuum feedback, and collision-aware motion planning. Integration with modern robotic middleware or standardized execution interfaces would also improve portability across industrial cells.

VI. Conclusion

This paper presented an automatic robot program generation method for robotic press brake sheet-metal handling. The method integrates orientation selection, suction grip planning, regrip handling, opposite-face bending support, trajectory generation, and pre-execution validation in a modular planning pipeline.

The research question was whether a rule-based program generator can reduce manual programming effort while maintaining feasible simulation execution for representative bending scenarios. The simulation results indicate that this is achievable within the stated assumptions. Executable robot programs were generated without manual waypoint definition, and adaptation after geometry or pose changes was reduced from manual reprogramming to automatic recalculation.

The contribution is a practical, deterministic planning architecture for structured robotic bending cells. The approach does not replace full offline programming, digital twin simulation, or advanced collision-aware motion planning. Instead, it

TABLE II
OBSERVED COMPARISON BETWEEN MANUAL AND AUTOMATED PROGRAMMING WORKFLOWS

Aspect	**Manual workflow**	**Automated workflow**
Waypoint definition	Manual	Automatic
Reprogramming after geometry change	Required	Not required
Adaptation to pose change	Manual adjustment	Automatic recalculation
Typical adaptation time per change	5–10 min	10–30 s
Regrip and flip handling	Manually sequenced	Planned by pipeline
Repeatability	Operator-dependent	Deterministic

provides a focused method for reducing operator dependency in flexible press brake automation where workpiece variation is present but the cell configuration remains controlled. XXXXXX

REFERENCES

[1] P. Bilancia, J. Schmidt, R. Raffaeli, M. Peruzzini, and M. Pellicciari, "An overview of industrial robots control and programming approaches," *Applied Sciences*, vol. 13, no. 4, p. 2582, 2023.

[2] S. K. Gupta, D. A. Bourne, K. H. Kim, and S. S. Krishnan, "Automated process planning for sheet metal bending operations," *Journal of Manufacturing Systems*, vol. 17, no. 5, pp. 338–360, 1998.

[3] S. Aomura and A. Koguchi, "Optimized bending sequences of sheet metal bending by robot," *Robotics and Computer-Integrated Manufacturing*, vol. 18, no. 1, pp. 29–39, 2002.

[4] F. Xu, Q. Jiang, L. Rong, P. Zhou, and J. Hu, "Structural model and dynamic analysis of six-axis cartesian coordinate robot for sheet metal bending," *International Journal of Advanced Robotic Systems*, vol. 16, no. 4, 2019.

[5] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, "Digital twin in industry: State-of-the-art," *IEEE Transactions on Industrial Informatics*, vol. 15, no. 4, pp. 2405–2415, 2019.

[6] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.

[7] J. Mahler, M. Matl, X. Liu, A. Li, D. Gealy, and K. Goldberg, "Dex-net 3.0: Computing robust vacuum suction grasp targets in point clouds using a new analytic model and deep learning," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, 2018, pp. 5620–5627.

[8] G. Wang, F. Xu, K. Zhou, and Z. Pang, "S-velocity profile of industrial robot based on nurbs curve and slerp interpolation," *Processes*, vol. 10, no. 11, p. 2195, 2022.